2026-10-21 MySQL基本查询：上

- 1. 表的增删改查与基础查询
  - 1.1 CRUD 总览
    - 1.1.1 什么是 CRUD
    - 1.1.2 准备一张学生表
- 2. 创建数据：INSERT
  - 2.1 INSERT 基本语法
    - 2.1.1 插入语句结构
    - 2.1.2 单行全列插入
    - 2.1.3 多行插入与指定列插入
  - 2.2 冲突时更新：ON DUPLICATE KEY UPDATE
    - 2.2.1 主键或唯一键冲突
    - 2.2.2 插入否则更新
  - 2.3 替换：REPLACE INTO
    - 2.3.1 REPLACE 的执行逻辑
    - 2.3.2 ON DUPLICATE KEY UPDATE 与 REPLACE 的区别
- 3. 读取数据：SELECT
  - 3.1 SELECT 基本语法
    - 3.1.1 查询语句结构
    - 3.1.2 准备考试结果表
  - 3.2 查询列
    - 3.2.1 全列查询
    - 3.2.2 指定列查询
  - 3.3 查询表达式与别名
    - 3.3.1 查询常量
    - 3.3.2 查询包含字段的表达式
    - 3.3.3 为查询结果指定别名
  - 3.4 查询结果去重
    - 3.4.1 DISTINCT 的基本作用
- 4. 条件筛选：WHERE
  - 4.1 WHERE 与运算符
    - 4.1.1 比较运算符
    - 4.1.2 逻辑运算符
  - 4.2 数值与范围筛选
    - 4.2.1 比较大小
    - 4.2.2 使用 AND 或 BETWEEN 查询闭区间

        - 4.2.3 使用 OR 或 IN 匹配多个离散值
    - 4.3 模糊匹配
        - 4.3.1 百分号 %
        - 4.3.2 下划线 _
    - 4.4 字段、表达式与别名
        - 4.4.1 比较两个字段
        - 4.4.2 在 WHERE 中使用表达式
        - 4.4.3 WHERE 不能直接使用 SELECT 别名
    - 4.5 逻辑条件组合
        - 4.5.1 AND 与 NOT
        - 4.5.2 使用括号明确 OR 与 AND 的组合
    - 4.6 NULL 的查询与比较
        - 4.6.1 判断 NULL
        - 4.6.2 NULL 不是 0，也不是空串
        - 4.6.3 = 与 <=> 对 NULL 的差异
- 5. 结果排序：ORDER BY
    - 5.1 ORDER BY 基本语法
        - 5.1.1 升序、降序与默认行为
        - 5.1.2 按单个字段排序
    - 5.2 NULL 值的排序位置
    - 5.3 多字段排序
        - 5.3.1 排序优先级
        - 5.3.2 使用表达式排序
        - 5.3.3 ORDER BY 可以使用 SELECT 别名
        - 5.3.4 WHERE 与 ORDER BY 结合
- 6. 分页：LIMIT
    - 6.1 LIMIT 的三种写法
        - 6.1.1 从表开头取 n 条
        - 6.1.2 使用 LIMIT s, n
        - 6.1.3 使用 LIMIT n OFFSET s
    - 6.2 分页查询示例
        - 6.2.1 每页 3 条进行分页
        - 6.2.2 分页与排序的关系
- 7. 本篇知识串联与易错点
    - 7.1 常用语句速查
        - 7.1.1 INSERT 与 SELECT 速查
        - 7.1.2 WHERE、ORDER BY 与 LIMIT 速查

- 7.2 高频错误与修正
  - 7.2.1 插入语句中的高频错误
  - 7.2.2 查询语句中的高频错误

1. 表的增删改查与基础查询

1.1 CRUD 总览

1.1.1 什么是 CRUD

对表中数据的常见操作可以概括为 CRUD：

英文	中文	对应 SQL	作用
Create	创建	`INSERT`	向表中插入数据
Retrieve	读取	`SELECT`	查询表中的数据
Update	更新	`UPDATE`	修改已有数据
Delete	删除	`DELETE`	删除已有数据

本篇先整理前半部分，重点包括：

1. 使用 `INSERT` 插入数据。
2. 使用 `SELECT` 查询列、表达式和去重结果。
3. 使用 `WHERE` 完成条件筛选。
4. 使用 `ORDER BY` 排序。
5. 使用 `LIMIT` 分页。

`UPDATE`、`DELETE`、聚合函数和 `GROUP BY` 等内容属于后续部分，不在本篇展开。

1.1.2 准备一张学生表

后续插入示例统一使用 `students` 表：

```
CREATE TABLE students (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    sn INT NOT NULL UNIQUE COMMENT '学号',
    name VARCHAR(20) NOT NULL,
```

```
    qq VARCHAR(20) UNIQUE
);
```

字段含义：

字段	含义	约束或默认行为
`id`	学生记录编号	主键、无符号整数、自增
`sn`	学号	非空、唯一
`name`	姓名	非空
`qq`	QQ 号	唯一，但允许为空

`AUTO_INCREMENT` 表示插入时如果省略 `id`，MySQL 会自动生成一个递增的编号。

2. 创建数据：INSERT

2.1 INSERT 基本语法

2.1.1 插入语句结构

```
INSERT [INTO] table_name
    [(column [, column] ...)]
    VALUES (value_list) [, (value_list)] ...

value_list: value [, value] ...
```

各部分含义：

部分	作用
`INSERT`	插入数据的关键字
`INTO`	可以省略，写上更易读
`table_name`	目标表名
`(column, ...)`	指定要插入的列；省略时表示全列插入
`VALUES (...)`	提供每一行对应的值
`, (...) ...`	一次插入多行

插入时要保证值的数量、顺序与表中列或指定列一致，并且数据类型兼容。

2.1.2 单行全列插入

不写列名时，`VALUES` 中的值必须与表的全部列按定义顺序一一对应：

```
-- 全列插入
INSERT INTO students VALUES (100, 10000, '唐三藏', NULL);

INSERT INTO students VALUES (101, 10001, '孙悟空', '11111');
```

插入后查看结果：

```
SELECT * FROM students;
```

id	sn	name	qq
100	10000	唐三藏	NULL
101	10001	孙悟空	11111

如果插入时省略自增的 `id`，必须明确指定其余列：

```
INSERT INTO students (sn, name, qq)
VALUES (123, '张飞', '123456');
```

由于没有指定 `id`，MySQL 会自动生成新的 `id`。

2.1.3 多行插入与指定列插入

一次插入多行数据，可以写成：

```
INSERT INTO students VALUES
    (13, 127, '张飞', '123498'),
    (14, 128, '许攸', '1233445'),
    (15, 129, '诸葛亮', '123459'),
    (16, 130, '刘备', '123488');
```

也可以一次向指定列插入多行。此时每个括号中的值必须与指定列的数量和顺序一致：

```
INSERT INTO students (id, sn, name) VALUES
    (102, 20001, '曹孟德'),
    (103, 20002, '孙仲谋');
```

因为本次没有指定 `qq`，新增记录的 `qq` 会使用列定义中的默认行为，这里为 `NULL`。

查询结果：

```
SELECT * FROM students;
```

id	sn	name	qq
100	10000	唐三藏	NULL
101	10001	孙悟空	11111
102	20001	曹孟德	NULL
103	20002	孙仲谋	NULL

2.2 冲突时更新：ON DUPLICATE KEY UPDATE

2.2.1 主键或唯一键冲突

如果插入的值与已有记录的主键或唯一键冲突，普通 `INSERT` 会失败。

```
-- id = 100 已经是唐三藏的主键，发生主键冲突
INSERT INTO students (id, sn, name)
VALUES (100, 10010, '唐大师');
```

执行结果类似：

```
ERROR 1062 (23000): Duplicate entry '100' for key 'PRIMARY'
```

如果学号重复，也会发生唯一键冲突：

```
INSERT INTO students (sn, name)
VALUES (20001, '曹阿瞞');
```

```
ERROR 1062 (23000): Duplicate entry '20001' for key 'sn'
```

2.2.2 插入否则更新

使用 `ON DUPLICATE KEY UPDATE` 后，发生冲突时可以把原本的插入动作改成更新动作：

```
INSERT INTO students (id, sn, name)
VALUES (100, 10010, '唐大师')
ON DUPLICATE KEY UPDATE
    sn = 10010,
    name = '唐大师';
```

执行结果有三种情况：

受影响行数	含义
`0`	发生冲突，但待更新的值与旧值完全相同
`1`	没有冲突，成功插入了一条新记录
`2`	发生冲突，旧记录被更新

可以使用 `ROW_COUNT()` 查看上一条语句影响的行数：

```
SELECT ROW_COUNT();
```

如果目标记录原本不存在，这条语句仍然会直接执行插入。

2.3 替换：REPLACE INTO

2.3.1 REPLACE 的执行逻辑

`REPLACE INTO` 的规则是：

1. 主键和唯一键都没有冲突时，直接插入。
2. 主键或唯一键冲突时，先删除发生冲突的旧记录，再插入新记录。

因此它的本质不是“原地修改”，而是“删除后重新插入”。

```
REPLACE INTO students (sn, name)
VALUES (20001, '曹阿瞞');
```

此时如果 `sn = 20001` 已存在，旧记录会先被删除，再将新记录插入。

针对唯一键冲突，也可以在 `qq` 相同的条件下替换整行：

```
REPLACE INTO students (sn, name, qq)
VALUES (140, '貂蝉', '9876543');

REPLACE INTO students (sn, name, qq)
VALUES (140, '吕布', '9876543');
```

第二次执行会因冲突删除第一次插入的记录，然后重新插入“吕布”这一行。

2.3.2 ON DUPLICATE KEY UPDATE 与 REPLACE 的区别

对比项	`ON DUPLICATE KEY UPDATE`	`REPLACE INTO`
冲突后的操作	更新指定列	删除整行后重新插入
未指定列	通常保留旧值	可能被默认值替代
自增主键	通常保持原记录	可能产生新的自增编号
适用场景	只想修改部分字段	希望用一整行覆盖冲突记录

---

3. 读取数据：SELECT

3.1 SELECT 基本语法

3.1.1 查询语句结构

```
SELECT
    [DISTINCT] {* | column [, column] ...}
    [FROM table_name]
    [WHERE ...]
    [ORDER BY column [ASC | DESC], ...]
    [LIMIT ...];
```

各子句的基本作用：

子句	作用
`SELECT`	选择要返回的列、常量或表达式
`DISTINCT`	对最终查询结果去重
`FROM`	指定数据来源表
`WHERE`	按条件筛选记录
`ORDER BY`	对结果排序
`LIMIT`	限制返回行数或进行分页

3.1.2 准备考试结果表

```
CREATE TABLE exam_result (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(20) NOT NULL COMMENT '同学姓名',
    chinese FLOAT DEFAULT 0.0 COMMENT '语文成绩',
    math FLOAT DEFAULT 0.0 COMMENT '数学成绩',
    english FLOAT DEFAULT 0.0 COMMENT '英语成绩'
);
```

插入测试数据：

```
INSERT INTO exam_result (name, chinese, math, english) VALUES
    ('唐三藏', 67, 98, 56),
    ('孙悟空', 87, 78, 77),
    ('猪悟能', 88, 98, 90),
    ('曹孟德', 82, 84, 67),
    ('刘玄德', 55, 85, 45),
    ('孙权', 70, 73, 78),
    ('宋公明', 75, 65, 30);
```

后续查询示例均以这张表为基础。

3.2 查询列

3.2.1 全列查询

```
SELECT * FROM exam_result;
```

结果包含表中的全部行和全部列：

id	name	chinese	math	english
1	唐三藏	67	98	56
2	孙悟空	87	78	77
3	猪悟能	88	98	90
4	曹孟德	82	84	67
5	刘玄德	55	85	45
6	孙权	70	73	78
7	宋公明	75	65	30

通常不建议在大量数据或正式查询中随意使用 `*`：

1. 查询的列越多，需要传输的数据量越大。
2. 返回无用列会增加处理负担。
3. 后续可能影响索引的使用。
4. 若表结构发生变化，结果列也可能随之变化。

3.2.2 指定列查询

可以只选择需要的列，并且指定列的顺序不必与建表顺序相同：

```
SELECT id, name, english FROM exam_result;
```

id	name	english
1	唐三藏	56
2	孙悟空	77
3	猪悟能	90
4	曹孟德	67
5	刘玄德	45
6	孙权	78
7	宋公明	30

3.3 查询表达式与别名

3.3.1 查询常量

`SELECT` 后面不仅可以写字段名，也可以写常量或表达式：

```
SELECT id, name, 10 FROM exam_result;
```

结果中每一行都会多出一列固定值 `10`。

3.3.2 查询包含字段的表达式

表达式可以只包含一个字段：

```
SELECT id, name, english + 10 FROM exam_result;
```

这会把每位同学的英语成绩加上 10 后返回，但不会修改原表中的数据。

表达式也可以同时包含多个字段：

```
SELECT id, name, chinese + math + english FROM exam_result;
```

查询结果是每位同学的总分：

id	name	chinese + math + english
1	唐三藏	221
2	孙悟空	242
3	猪悟能	276
4	曹孟德	233
5	刘玄德	185
6	孙权	221
7	宋公明	170

```
mysql> select name , math+chinese+english from exam_result;
+-----------+-----------------------+
| name      | math+chinese+english  |
+-----------+-----------------------+
| 唐三藏    |                   221 |
| 孙悟空    |                   242 |
| 猪悟能    |                   276 |
| 曹孟德    |                   233 |
| 刘玄德    |                   185 |
| 孙权      |                   221 |
| 宋公明    |                   170 |
+-----------+-----------------------+
7 rows in set (0.00 sec)
```

3.3.3 为查询结果指定别名

当表达式结果较长或没有明确列名时，可以使用别名改善可读性：

```sql
SELECT column [AS] alias_name
FROM table_name;
```

`AS` 可以省略：

```sql
SELECT
    id,
    name,
    chinese + math + english AS 总分
FROM exam_result;
```

也可以写成：

```
SELECT name 姓名, math + chinese + english 总分
FROM exam_result;
```

结果表头会显示为“姓名”和“总分”。

3.4 查询结果去重

3.4.1 DISTINCT 的基本作用

原始数学成绩中存在两个 `98`：

```
SELECT math FROM exam_result;
```

使用 `DISTINCT` 后，只保留不同的值：

```
SELECT DISTINCT math FROM exam_result;
```

查询结果：

math
98
78
84
85
73
65

`DISTINCT` 作用于最终返回的结果行，不会修改原表中的数据。

4. 条件筛选：WHERE

4.1 WHERE 与运算符

4.1.1 比较运算符

运算符	说明
`>`、`>=`、`<`、`<=`	大于、大于等于、小于、小于等于
`=`	等于；对 `NULL` 不安全，例如 `NULL = NULL` 的结果是 `NULL`
`<=>`	等于；对 `NULL` 安全，例如 `NULL <=> NULL` 的结果是 `TRUE(1)`
`!=`、`<>`	不等于
`BETWEEN a0 AND a1`	范围匹配，闭区间 `[a0, a1]`
`IN (option, ...)`	值等于选项中的任意一个时匹配
`IS NULL`	判断是否为 `NULL`
`IS NOT NULL`	判断是否不为 `NULL`
`LIKE`	模糊匹配，`%` 表示任意多个字符，`_` 表示任意一个字符

4.1.2 逻辑运算符

运算符	说明
`AND`	多个条件都为 `TRUE(1)` 时，结果才为 `TRUE(1)`
`OR`	任意一个条件为 `TRUE(1)` 时，结果为 `TRUE(1)`
`NOT`	对条件取反；条件为 `TRUE(1)` 时，结果为 `FALSE(0)`

`AND` 的优先级高于 `OR`。当查询同时包含二者时，应使用括号明确条件的组合关系。

4.2 数值与范围筛选

4.2.1 比较大小

查询英语不及格的同学及英语成绩：

```
SELECT name, english
FROM exam_result
WHERE english < 60;
```

name	english
唐三藏	56
刘玄德	45
宋公明	30

4.2.2 使用 AND 或 BETWEEN 查询闭区间

查询语文成绩在 `[80, 90]` 之间的同学：

```
SELECT name, chinese
FROM exam_result
WHERE chinese >= 80 AND chinese <= 90;
```

等价的区间写法：

```
SELECT name, chinese
FROM exam_result
WHERE chinese BETWEEN 80 AND 90;
```

两种写法都会返回：

name	chinese
孙悟空	87
猪悟能	88
曹孟德	82

`BETWEEN a0 AND a1` 包含边界值 `a0` 和 `a1`。

4.2.3 使用 OR 或 IN 匹配多个离散值

查询数学成绩是 58、59、98、99 分之一的同学：

```sql
SELECT name, math
FROM exam_result
WHERE math = 58
   OR math = 59
   OR math = 98
   OR math = 99;
```

更简洁的写法：

```sql
SELECT name, math
FROM exam_result
WHERE math IN (58, 59, 98, 99);
```

查询结果：

name	math
唐三藏	98
猪悟能	98

4.3 模糊匹配

4.3.1 百分号 `%`

`%` 可以匹配任意多个字符，包括 0 个字符。

查询所有姓孙的同学：

```sql
SELECT name
FROM exam_result
WHERE name LIKE '孙%';
```

结果包括“孙悟空”和“孙权”。

4.3.2 下划线 `_`

`_` 严格匹配一个任意字符。

查询姓孙且姓名只有两个字的同学：

```
SELECT name
FROM exam_result
WHERE name LIKE '孙_';
```

结果只有：

name
孙权

4.4 字段、表达式与别名

4.4.1 比较两个字段

查询语文成绩高于英语成绩的同学：

```
SELECT name, chinese, english
FROM exam_result
WHERE chinese > english;
```

name	chinese	english
唐三藏	67	56
孙悟空	87	77
曹孟德	82	67
刘玄德	55	45
宋公明	75	30

`WHERE` 两侧可以是字段，也可以是常量或表达式。

4.4.2 在 WHERE 中使用表达式

查询总分在 200 分以下的同学：

```
SELECT
    name,
    chinese + math + english AS 总分
FROM exam_result
WHERE chinese + math + english < 200;
```

name	总分
刘玄德	185
宋公明	170

4.4.3 WHERE 不能直接使用 SELECT 别名

下面的写法不成立：

```
SELECT
    name,
    chinese + math + english AS total
FROM exam_result
WHERE total < 200;
```

原因是执行逻辑上先完成 `WHERE` 筛选，再形成最终查询列和别名。因此 `WHERE` 中不能直接引用 `SELECT` 子句新定义的别名。

正确写法仍是重复表达式：

```
SELECT
    name,
    chinese + math + english AS total
FROM exam_result
WHERE chinese + math + english < 200;
```

4.5 逻辑条件组合

4.5.1 AND 与 NOT

查询语文成绩大于 80 且不姓孙的同学：

```
SELECT name, chinese
FROM exam_result
WHERE chinese > 80
  AND name NOT LIKE '孙%';
```

name	chinese
猪悟能	88
曹孟德	82

4.5.2 使用括号明确 OR 与 AND 的组合

要求命中任一条件：

1. 姓孙且名字为两个字。
2. 或者总分大于 200，同时语文成绩小于数学成绩，并且英语成绩大于 80。

```
SELECT
    name,
    chinese,
    math,
    english,
    chinese + math + english AS 总分
FROM exam_result
WHERE name LIKE '孙_'
   OR (
        chinese + math + english > 200
        AND chinese < math
        AND english > 80
   );
```

name	chinese	math	english	总分
猪悟能	88	98	90	276
孙权	70	73	78	221

括号不是可有可无的装饰。没有括号时，`AND` 会先结合，条件含义可能改变。

4.6 NULL 的查询与比较

4.6.1 判断 NULL

查看 `students` 表中的原始数据：

id	sn	name	qq
100	10010	唐大师	NULL
101	10001	孙悟空	11111
103	20002	孙仲谋	NULL
104	20001	曹阿瞒	NULL

查询 QQ 号已知的同学：

```
SELECT name, qq
FROM students
WHERE qq IS NOT NULL;
```

结果只有：

name	qq
孙悟空	11111

查询 QQ 号为空的同学应使用：

```
SELECT name, qq
FROM students
WHERE qq IS NULL;
```

4.6.2 `NULL` 不是 0，也不是空串

`NULL` 表示“未知或不存在”，0 是一个确定的数字，`''` 是一个长度为零的字符串。三者不能混为一谈。

```
SELECT * FROM test WHERE name IS NULL;
```

空串与 `NULL` 不相同，因此判断空串要使用与空串对应的条件，不能把它当成 `NULL`。

4.6.3 `=` 与 `<=>` 对 NULL 的差异

```
SELECT NULL = NULL, NULL = 1, NULL = 0;
```

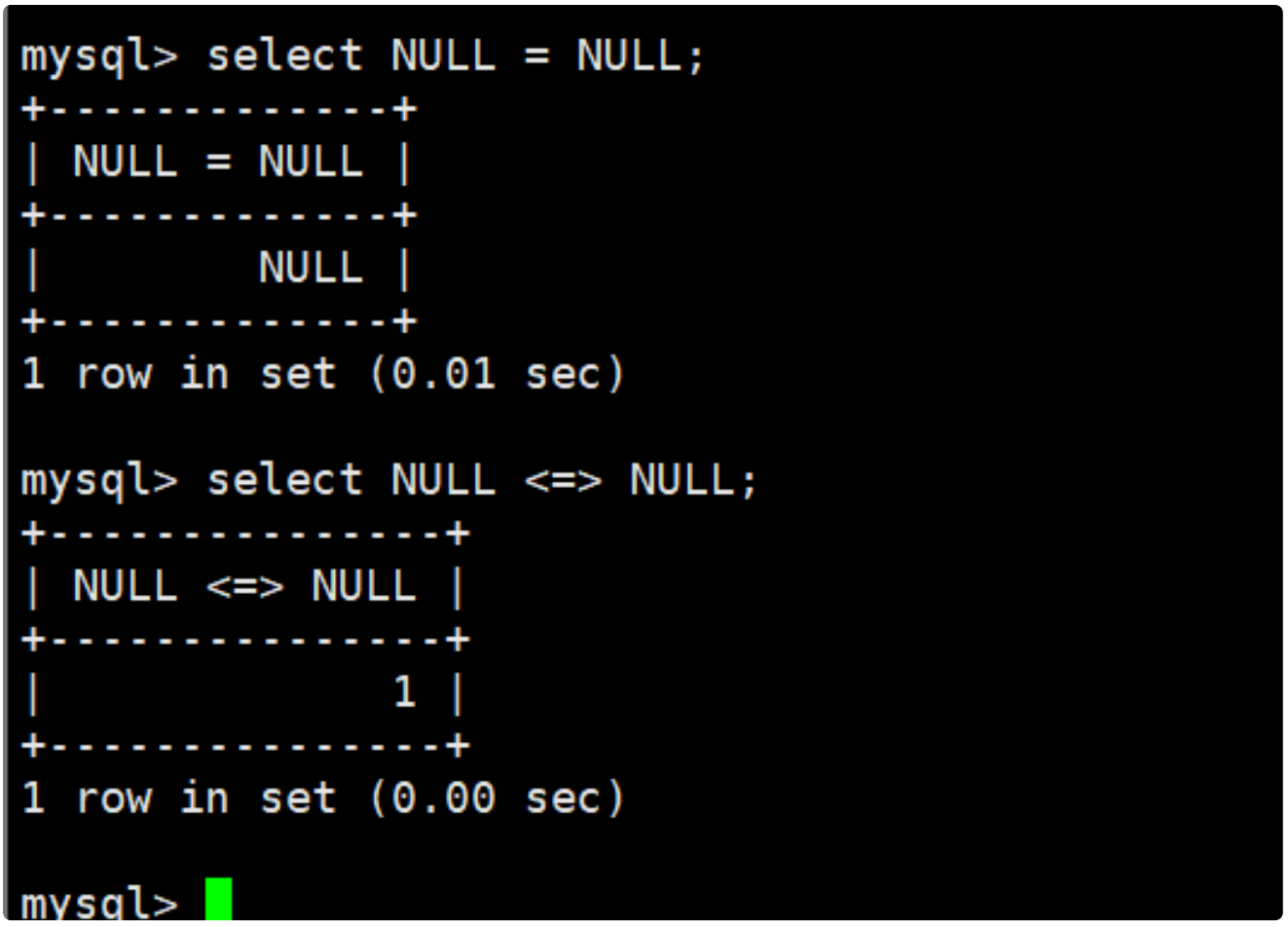
结果都是 `NULL`，不是 `1` 或 `0`：

NULL = NULL	NULL = 1	NULL = 0
NULL	NULL	NULL

使用 NULL 安全等于 `<=>`：

```
SELECT NULL <=> NULL, NULL <=> 1, NULL <=> 0;
```

NULL <=> NULL	NULL <=> 1	NULL <=> 0
1	0	0



结论：

1. 判断是否为 `NULL`，优先使用 `IS NULL` 或 `IS NOT NULL`。
2. `NULL = NULL` 的结果仍是 `NULL`，不能据此判断相等。
3. `NULL <=> NULL` 的结果为 `1`，它是 NULL 安全比较。
4. `NULL` 与 0、空串不是同一个概念。

5. 结果排序：ORDER BY

5.1 ORDER BY 基本语法

5.1.1 升序、降序与默认行为

```
SELECT ...
FROM table_name
[WHERE ...]
ORDER BY column [ASC | DESC], [...];
```

关键字	作用
`ASC`	升序，从小到大，默认行为
`DESC`	降序，从大到小

如果没有 `ORDER BY`，数据库返回记录的顺序是未定义的，不能把当前看到的顺序当成稳定规则。

5.1.2 按单个字段排序

按数学成绩升序显示：

```
SELECT name, math
FROM exam_result
ORDER BY math ASC;
```

按数学成绩降序显示：

```
SELECT name, math
FROM exam_result
ORDER BY math DESC;
```

5.2 NULL 值的排序位置

在 MySQL 的该排序规则中，`NULL` 可视为比任何普通值都小：

```
SELECT name, qq
FROM students
ORDER BY qq ASC;
```

`NULL` 会出现在最上面。

```
SELECT name, qq
FROM students
ORDER BY qq DESC;
```

`NULL` 会出现在最下面。

5.3 多字段排序

5.3.1 排序优先级

查询各科成绩，依次按数学降序、英语升序、语文升序显示：

```
SELECT name, math, english, chinese
FROM exam_result
ORDER BY math DESC, english ASC, chinese ASC;
```

多字段排序的优先级由书写顺序决定：

1. 先按 `math DESC` 排序。
2. 数学成绩相同时，再按 `english ASC` 排序。
3. 数学和英语都相同时，再按 `chinese ASC` 排序。

如果省略 `ASC`，默认仍为升序：

```
SELECT name, math, english, chinese
FROM exam_result
ORDER BY math DESC, english, chinese;
```

建议把 `ASC` 或 `DESC` 写全，便于阅读和维护。

5.3.2 使用表达式排序

查询总分由高到低：

```
SELECT
    name,
    chinese + english + math AS total
FROM exam_result
ORDER BY chinese + english + math DESC;
```

结果：

name	total
猪悟能	276
孙悟空	242
曹孟德	233
唐三藏	221
孙权	221
刘玄德	185
宋公明	170

5.3.3 ORDER BY 可以使用 SELECT 别名

```
SELECT
    name,
    chinese + english + math AS 总分
FROM exam_result
ORDER BY 总分 DESC;
```

这里可以使用别名，是因为 `ORDER BY` 的语义发生在查询列已经形成之后。

对比记忆：

子句	能否使用 SELECT 新定义的别名	原因
`WHERE`	不能	先筛选原始记录，再形成最终查询列和别名
`ORDER BY`	可以	查询列和别名已经形成后再排序

5.3.4 WHERE 与 ORDER BY 结合

查询姓孙或者姓曹的同学及其数学成绩，并按数学成绩降序：

```
SELECT name, math
FROM exam_result
WHERE name LIKE '孙%'
   OR name LIKE '曹%'
ORDER BY math DESC;
```

name	math
曹孟德	84
孙悟空	78
孙权	73

6. 分页：LIMIT

6.1 LIMIT 的三种写法

6.1.1 从表开头取 n 条

```
SELECT ...
FROM table_name
[WHERE ...]
[ORDER BY ...]
LIMIT n;
```

例如，从结果开头连续读取 3 行：

```
SELECT * FROM exam_result LIMIT 3;
```

6.1.2 使用 `LIMIT s, n`

```
SELECT ...
FROM table_name
[WHERE ...]
[ORDER BY ...]
LIMIT s, n;
```

含义是从下标 `s` 开始，读取 `n` 条。起始下标从 0 开始：

```
SELECT * FROM exam_result LIMIT 3, 3;
```

表示先跳过 3 条，再取 3 条。

6.1.3 使用 `LIMIT n OFFSET s`

```
SELECT ...
FROM table_name
[WHERE ...]
[ORDER BY ...]
LIMIT n OFFSET s;
```

它表示从偏移量 `s` 开始读取 `n` 条。这种写法语义更清楚，实际使用中建议采用。

6.2 分页查询示例

6.2.1 每页 3 条进行分页

对未知表查询时，可以先用 `LIMIT 1` 查看少量数据，避免表过大时查询全部内容给数据库造成压力。

按 `id` 排序，每页 3 条：

第 1 页：

```
SELECT id, name, math, english, chinese
FROM exam_result
ORDER BY id
LIMIT 3 OFFSET 0;
```

第 2 页：

```
SELECT id, name, math, english, chinese
FROM exam_result
ORDER BY id
LIMIT 3 OFFSET 3;
```

第 3 页：

```
SELECT id, name, math, english, chinese
FROM exam_result
```

```
ORDER BY id
LIMIT 3 OFFSET 6;
```

页码	偏移量	返回结果
第 1 页	0	`id` 为 1、2、3 的记录
第 2 页	3	`id` 为 4、5、6 的记录
第 3 页	6	`id` 为 7 的记录

最后一页不足 3 条时仍然正常返回，不会报错。

6.2.2 分页与排序的关系

分页前应先通过 `ORDER BY` 明确结果顺序。否则每一页内部以及页与页之间的顺序都不稳定，连续翻页可能出现重复或遗漏。

可以按下面的顺序理解本篇各子句的作用：

1. `FROM` 确定数据来源。
2. `WHERE` 筛选符合条件的记录。
3. `SELECT` 形成查询列和别名。
4. `DISTINCT` 对结果去重。
5. `ORDER BY` 对结果排序。
6. `LIMIT` 最后截取要返回的行。

因此，`LIMIT` 的本质不是“控制数据库读取或排序的范围”，而是对已经形成的结果限制最终显示数量；若需要稳定分页，必须配合 `ORDER BY`。

7. 本篇知识串联与易错点

7.1 常用语句速查

7.1.1 INSERT 与 SELECT 速查

目标	示例
全列单行插入	`INSERT INTO t VALUES (1, 'A');`

目标	示例
指定列单行插入	`INSERT INTO t (name) VALUES ('A');`
多行插入	`INSERT INTO t VALUES (1, 'A'), (2, 'B');`
冲突时更新	`INSERT ... ON DUPLICATE KEY UPDATE ...`
冲突时删后插入	`REPLACE INTO t ...`
全列查询	`SELECT * FROM t;`
指定列查询	`SELECT name, score FROM t;`
表达式与别名	`SELECT name, score + 10 AS new_score FROM t;`
结果去重	`SELECT DISTINCT score FROM t;`

7.1.2 WHERE、ORDER BY 与 LIMIT 速查

目标	示例
数值比较	`WHERE score >= 60`
闭区间	`WHERE score BETWEEN 60 AND 90`
枚举值	`WHERE score IN (80, 90)`
模糊匹配	`WHERE name LIKE '孙%'`
判断 NULL	`WHERE qq IS NULL`
组合条件	`WHERE a > 1 AND (b < 2 OR c = 3)`
升序	`ORDER BY score ASC`
降序	`ORDER BY score DESC`
多字段排序	`ORDER BY math DESC, english ASC`
取前 n 条	`LIMIT n`
分页	`LIMIT n OFFSET s`

7.2 高频错误与修正

7.2.1 插入语句中的高频错误

1. 全列插入时，值的数量与表的列数不一致。
2. 指定列插入时，值的顺序与指定列的顺序不一致。
3. 把允许为 `NULL` 的字段误写成字符串 `'NULL'`。
4. 发生主键或唯一键冲突时仍使用普通 `INSERT`。
5. 误以为 `REPLACE` 是普通更新，忽略它会删除旧行后重新插入。

6. 省略自增 `id` 时，没有明确指定其余要插入的列。

7.2.2 查询语句中的高频错误

1. 在正式查询中无条件使用 `SELECT *`，造成无效数据传输。
2. 认为没有 `ORDER BY` 时结果一定按插入顺序返回。
3. 在 `WHERE` 中直接引用 `SELECT` 新定义的别名。
4. 混淆 `LIKE '%'` 与 `LIKE '_'` 的匹配含义。
5. 使用 `NULL = NULL` 或 `qq = NULL` 判断 NULL。
6. 把 `NULL`、0 和空串当作同一个值。
7. 同时使用 `AND`、`OR` 时没有用括号明确优先级。
8. 分页时没有 `ORDER BY`，导致翻页结果不稳定。
9. 把 `LIMIT 3, 3` 误解为“从第 3 行开始”，实际起始下标为 0。
10. 把 `LIMIT` 当作改变原表数据或排序过程的操作。